

SIMATS ENGINEERING ASSESSMENT TOOL

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO2: Design Finite Automata DFA, NFA, ϵ -NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 10%

QUESTIONS: 5

Total Marks:25

Duration : 1 Hour

Application Based Question

Code of Conduct:

I A. Manoj Reddy ¹⁹²³¹¹¹⁷¹ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Manoj

SIMATS ENGINEERING ASSESSMENT TOOL

1. In modern digital systems such as embedded security devices, ATM machines, and authentication modules, password validation must be efficient and reliable. Design a **Deterministic Finite Automaton (DFA)** over the alphabet $\Sigma = \{0,1\}$ to validate passwords that **end with the pattern "01"**. Further, **minimize the DFA** and analyze how the optimized design improves system efficiency in real-world hardware implementations.
2. In web applications, validating user input is essential to ensure correctness, security, and performance. Design a regular expression to validate a specific input pattern "aaaabb" over the alphabet $\Sigma = \{a, b\}$. Further, analyze how regular languages and regular expressions are used to efficiently validate user inputs in real-world web applications.
3. In modern computing systems such as intrusion detection systems, compilers, and real-time monitoring tools, pattern recognition plays a critical role. Demonstrate how the regular expression a^*b^* can be converted into an equivalent finite automaton (FA). Further, explain why regular expressions and finite automata are equivalent models and how they are effectively used in real-time detection systems.
4. In compiler design, not all patterns can be recognized using finite automata. Using the pumping lemma, demonstrate why the language $L = \{ a^n b^n \mid n \geq 0 \}$ cannot be recognized by any finite automaton. Further, analyze the implications of this limitation in the design of modern compilers.
5. Modern systems such as intrusion detection, spam filtering, and input validation often require combining multiple pattern-matching rules. Explain how the **closure properties of regular languages** enable the combination of multiple detection rules. Further, justify why the resulting language after applying these operations **remains regular**, ensuring efficient implementation using finite automata.

DFA for password ending with 01

Given:

Alphabet: $\Sigma = \{0, 1\}$

Required pattern: 01 (The DFA must accept all String end with 01)

q_0 :- initial state

q_1 :- the last Symbol 0

q_2 :- the String 1

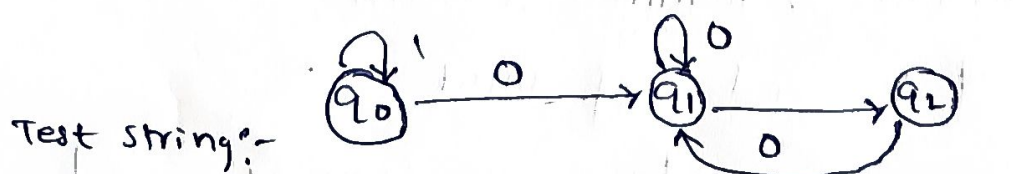
from q_0 :-

On 0 $\rightarrow q_1$	q_1 :-	On 0 $\rightarrow q_1$
On 1 $\rightarrow q_0$	On 1 $\rightarrow q_2$	On 1 $\rightarrow q_0$

Transition table:-

Current state	input 0	input 1
q_0	q_1	q_0
q_1	q_1	q_2
q_2	q_1	q_0

DFA representation:-



String	Result
01	Accepted
101	Accepted
0001	Accepted
1101	Accepted

String	Result
10	Rejected
011	Rejected
111	Rejected

∴ The minimized DFA has 5 states and effectively validate pass ending with 01

2) Regular expression:-

" a a a a b b "

Given :-

$\Sigma = \{a, b\}$

equivalent notation :- $a^4 b^2$

Input	Result
aaaaabb	Accepted
aaqab	Rejected
aaabbb	Rejected
aaaqab	Rejected
aaaabbbb	Rejected
aabbaab	Rejected

A web application can use:-
" a a a a b b \$

here x → beginning of input

aaaaabb → required pattern

\$ → end of input

Regular expression provide a compact and efficient method for validating regular pattern

Conversion of Regular Expression $a^* b^*$ into FA

given:-

$a^* b^*$

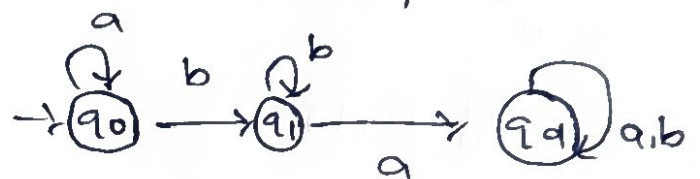
Example: accepted string

$\epsilon, a, aa, aaabbb$

$b, bb, ab, aabb$

three states: q_0, q_1, q_d

FA diagram



Transition table:-

State	a	b
q_0	q_0	q_1
q_1	q_d	q_1
q_d	q_d	q_d

$a^* b^*$ can be efficiently recognized by a finite

Automata

4) Pumping lemma: ~~prove~~ $L = \{ a^n b^n \mid n \geq 1 \}$ is not regular

language: $L = \{ a^n b^n \mid n \geq 1 \}$

example:-

$\epsilon, ab, aabb, aaabbb, aaaabbbb \dots$

Selecting a string

Choose $w = a^p b^p$

clearly $w \in L$

$|w| = 2p \quad ; \quad w = xyz$

$xy^0z = xz$, No of a 's: $p-k$

Since $P \rightarrow K \neq P$

This is Contradiction

$\therefore$ our assumption is false

$L = \{ a^n b^n \mid n \geq a \}$ is not regular language.

5)

Given :-

if L_1, L_2

then $L_1 \cup L_2$

L_1 : malware pattern

L_2 : Phishing pattern

$L = L_1 \cup L_2$

given that,

if L_1, L_2 then it is regular

L^* , $L = \{a, b\}$

$L^* = \{ \epsilon, ab, abab, ababab \}$

$\therefore (L_1 \cup L_2 \cup L_3) \therefore$ is regular

DFA₁ = $(Q_1, \Sigma, \delta_1, q_1, F_1)$

DFA₂ = $(Q_2, \Sigma, \delta_2, q_2, F_2)$

$\therefore F = (F_1 \times Q_2) \cup (Q_1 \times F_2)$

SIMATS ENGINEERING ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

88/100